

API SECURITY ASSESSMENT

ReNew Solar Marketing Website

API Surface Assessment & Pre-Implementation Threat Model

CONFIDENTIAL. Distribute only to personnel responsible for the systems described. **Please read Section 1 first:** this application has no application API. The scope determination, and what this report delivers instead, are set out there.

TARGET

ReNew Solar Marketing Site

ASSESSMENT DATE

17 August 2026

CODE STATE

a4f4020 + remediation

FRAMEWORK

Next.js 16.2.11 · static export

APPLICATION APIS FOUND

0 — verified, Section 2

DATA ENDPOINTS FOUND

49 read-only static artifacts

STANDARDS

OWASP API Security Top 10:2023

REPORT VERSION

1.0 — Final

1 Executive Summary

SCOPE REALITY — READ BEFORE ANYTHING ELSE

This application exposes no API. It has no route handlers, no middleware, no server actions, no backend of any kind, and its client code makes no network requests whatsoever. This was verified independently in the VAPT report of 17 August 2026 and **re-verified from scratch for this assessment** against the remediated codebase — which now runs a newer framework version and two additional generated routes.

The enquiry-submission endpoint that would have been the site's first real API was **explicitly deferred by the client** during remediation on the same date. It does not exist yet.

An API security assessment with no API to assess could be padded into something impressive-looking. It will not be here. Instead this report does three things that are genuinely useful:

1. **Proves the absence** (Section 2) with fresh evidence, so the conclusion is auditable rather than asserted — and so a future change that introduces an API is immediately visible as a deviation from a documented baseline.
2. **Assesses the read-only data plane that does exist** (Sections 3–4). The site publishes 49 machine-readable endpoints — sitemap, robots, security.txt, and 46 React Server Component payload files. These are not an API in the conventional sense, but they are HTTP endpoints returning structured data to non-browser consumers, and they were tested as such. **One real defect was found and fixed during this assessment.**
3. **Specifies the security controls for the API you will build** (Section 6). The deferred enquiry endpoint is mapped against OWASP API Security Top 10:2023 with concrete, implementable controls. This is the section with lasting value: it is far cheaper to build these in than to retrofit them after a penetration test finds them missing.

Findings by severity



THE ONE REAL FINDING — AND ITS PROVENANCE

API-SEC-01 is a genuine defect: the sitemap endpoint interpolated an unvalidated build-time environment variable directly into XML, and Next.js does not escape the `<loc>` element. A malformed origin produced a sitemap that no crawler could parse.

In the interest of straight reporting: **this defect was introduced by the remediation work carried out earlier the same day**, in code written to close VAPT-06. It was found by testing that new code rather than assuming it was sound, and it was **fixed and re-verified during this assessment**. It is documented in full in Section 4 rather than quietly corrected.

Overall assessment

No API attack surface exists to compromise. The data endpoints that do exist are read-only, unauthenticated by design, contain nothing not already public in the HTML (verified in Section 3.3), and accept no input. The realistic API risk to this property today is **nil** — and it will stay that way exactly until the enquiry endpoint is built, at which point Section 6 becomes the relevant document.

2 Verification of API Absence

A negative finding is only worth as much as the evidence behind it. Six independent checks were run across source, build output and runtime. All six are negative.

2.1 Source-level checks

```
# 1. No route handlers anywhere in the source tree

(no results)

# 2. No app/api directory, no middleware

(no results)

# 3. No Server Actions

no server actions

# 4. No outbound calls of any kind from client code

none — app makes no network calls
```

2.2 Build-level check

Every route in the production build is marked `○ (Static)` — prerendered to a file. A route capable of serving an API would be marked dynamic or as a function:

```
○ (Static) prerendered as static content ← all 9 routes, no exceptions
```

2.3 Runtime check

Confirmed against the live production host during the DAST engagement: a request to a plausible API path returns the static 404 document, not an API error.

```
404
```

2.4 A nuance worth recording

THE TWO ROUTE HANDLERS THAT DO EXIST — AND WHY THEY ARE NOT AN API

Remediation added `src/app/robots.ts` and `src/app/sitemap.ts`. In Next.js terms these are *technically* Route Handlers, and a naive scan for "route handlers = API surface" would flag them.

They are not an API. Both carry `export const dynamic = "force-static"` — mandatory under `output: "export"` — which means they execute **once at build time** and emit a flat file. Nothing of them survives into the deployment; there is no handler at runtime to reach. They accept no request, read no request context, and cannot be invoked with parameters. They are build scripts that happen to use a route-shaped file convention.

That said, they *do* produce two of the endpoints assessed in Section 3, and one of them carried the defect reported as API-SEC-01. Build-time code that writes published output deserves the same scrutiny as runtime code — which is the reason this nuance is documented rather than waved past.

2.5 Conclusion

API CAPABILITY	PRESENT	CONSEQUENCE FOR THIS ASSESSMENT
Runtime request handling	NO	No endpoint can be invoked
Authentication / authorization	NO	API1, API2, API3, API5 not applicable
Request body / parameter parsing	NO	No injection or mass-assignment surface
Data mutation	NO	All endpoints read-only; nothing to tamper with
Rate-limitable resource consumption	NO	CDN-served static files; API4 not applicable
Outbound / third-party API consumption	NO	API10 not applicable — zero network calls
Read-only static data endpoints	YES	Assessed in Section 3

3 The Read-Only Data Plane

While no API exists, the site does publish machine-readable endpoints — files served over HTTP whose consumers are crawlers, security researchers and the application's own client runtime rather than human readers. They accept no input and return fixed content, but they are the closest thing to an API surface this property has, so they were assessed as one.

3.1 Endpoint inventory

ENDPOINT	COUNT	CONTENT TYPE	PURPOSE & CONSUMER
/sitemap.xml	1	application/xml	Crawler index — 6 URLs. New (remediation)
/robots.txt	1	text/plain	Crawler directives. New (remediation)
/.well-known/security.txt	1	text/plain	RFC 9116 disclosure channel. New (remediation)
/*.txt	14	text/plain	Per-route React Server Component payloads
/**/*.__next.*.txt	32	text/plain	RSC tree, head and page segments for client navigation
Total	49	—	All read-only, all unauthenticated by design

3.2 Assessment against API-relevant controls

CONTROL	RESULT	OBSERVATION
Input handling	N/A	No endpoint accepts a parameter, body, or header that alters its response
Authentication	BY DESIGN	All content is public marketing material; authentication would serve no purpose
Authorization	N/A	No per-user or per-object data exists to authorize against
Rate limiting	PLATFORM	CDN-served; platform bot-mitigation observed active during DAST
Content-type correctness	CORRECT	XML served as <code>application/xml</code> , text as <code>text/plain</code> — verified per endpoint
MIME-sniffing protection	PENDING	<code>X-Content-Type-Options: nosniff</code> now in <code>vercel.json</code> ; effective on deploy
Sensitive data exposure	CLEAN	Verified — see 3.3
Injection into generated output	FINDING	API-SEC-01 — found and fixed during this assessment
Response integrity	VALID	security.txt RFC 9116-compliant; all 6 sitemap URLs resolve to real routes

3.3 Do the RSC payloads leak anything?

The 46 React Server Component payload files are the least obvious endpoints on the site and the most plausible place for accidental disclosure — they contain the serialised component tree, which in some applications carries props never rendered visibly. Both the payload and the corresponding HTML were scanned for credential-shaped strings and compared:

```
none
none
none
```

RESULT

No disclosure. The payloads contain nothing absent from the rendered HTML, and the only hosts referenced are the public ReNew and social properties already linked on the page. The pattern set searched covered `api_key`, `secret`, `token`, `password`, `bearer`, `authorization`, `internal`, `staging`, `admin`, `.env`, `AKIA` and `sk_live` — zero matches in either file.

4 Findings

API-SEC-01 **Unvalidated environment variable injected into sitemap XML**

LOW

CWE	OWASP API	STATUS	FILE
CWE-91 — XML Injection	API8:2023 — Misconfiguration	Fixed & verified	src/lib/site.ts

DESCRIPTION

The sitemap endpoint built its URLs by interpolating `NEXT_PUBLIC_SITE_URL` directly into the output. Next.js's sitemap generator writes the `<loc>` element **without XML-escaping its contents**, so any metacharacter in that variable passes through raw into the published file.

```
// src/lib/site.ts — as originally written during remediation
export const SITE_URL =
  process.env.NEXT_PUBLIC_SITE_URL ?? // interpolated verbatim
  (process.env.NEXT_PUBLIC_VERCEL_URL
    ? `https://${process.env.NEXT_PUBLIC_VERCEL_URL}`
    : "https://re-new-mocha.vercel.app");

// src/app/sitemap.ts
url: `${SITE_URL}${path}`, // → emitted into <loc> unescaped
```

EVIDENCE — REPRODUCTION

```
# builds "successfully"

<loc>https://evil.test/"><script>alert(1)</script>&foo/</loc>

True
True
NO — not well-formed (invalid token): line 4, column 54
```

IMPACT

Rated **Low**, and the reasoning matters. Exploiting this requires the ability to set a build-time environment variable — meaning access to the Vercel project settings or the CI configuration. Anyone holding that access can already alter the deployment far more directly, so this is not a privilege-escalation path.

The realistic impact is **silent breakage**: a malformed sitemap is rejected by every crawler that parses it, so the SEO control added to close VAPT-06 would quietly stop working with no error at build time — the build exits `0`. Secondary impact is content injection into a machine-consumed feed. It is *not* a browser XSS vector: the file is served as `application/xml`, and in any case a malformed document is rejected before script could be considered.

The more general lesson is the one worth keeping: **build-time code that writes published output is part of the attack surface**, and an unvalidated environment variable is untrusted input even when the person setting it is trusted — because typos are far more likely than attacks.

REMEDIATION APPLIED

The origin is now parsed and normalised rather than interpolated. Using `URL.origin` is what does the real work: it discards path, query, fragment and credentials, which is precisely where XML metacharacters would live.

```
function normaliseOrigin(candidate: string | undefined): string | null {
  if (!candidate) return null;
  const withScheme = /^https?:\/\//i.test(candidate) ? candidate : `https://${candidate}`;

  let parsed: URL;
  try { parsed = new URL(withScheme); } catch { return null; }

  if (parsed.protocol !== "https:" && parsed.protocol !== "http:") return null;

  const origin = parsed.origin; // drops path/query/fragment – and the metacharacters
  if (origin === "null" || /[<>&"]/.test(origin)) return null; // belt and braces
  return origin;
}

export const SITE_URL =
  normaliseOrigin(process.env.NEXT_PUBLIC_SITE_URL) ??
  normaliseOrigin(process.env.NEXT_PUBLIC_VERCEL_URL) ??
  FALLBACK_ORIGIN;
```

VERIFICATION

```
# Same hostile value as the original reproduction
https://evil.test/
False
YES

# Junk values fall back safely rather than emitting garbage

https://re-new-mocha.vercel.app/ (rejected)
https://re-new-mocha.vercel.app/ (rejected)
https://re-new-mocha.vercel.app/ (fallback)
https://renew.com/ (bare host upgraded)
```

Post-fix: `tsc` clean, ESLint clean, build produces 9 static routes, sitemap valid with all 6 URLs resolving to real build artifacts.

API-SEC-02 **React Server Component payloads publicly enumerable**

INFO

CWE	OWASP API	SENSITIVE DATA	CROSS-REF
CWE-200 — Information Exposure	API9:2023 — Inventory Management	None — verified §3.3	DAST-08

DESCRIPTION & IMPACT

46 `.txt` files containing serialised RSC payloads are retrievable without restriction, including an `__next._full.txt` per route holding the complete component tree. They are a normal product of the App Router static export, not a misconfiguration, and Section 3.3 confirms they disclose nothing beyond the HTML.

Recorded here for an inventory reason rather than a confidentiality one. This is **API9:2023 — Improper Inventory Management** in its mildest form: 46 of the site's 49 machine-readable endpoints are undocumented, non-obvious, and unlikely to appear on anyone's mental map of what the site publishes. That matters less for their current contents than for what it implies about future changes — content added to a page appears here automatically, in plain text, whether or not anyone considered that endpoint.

RECOMMENDATION

No action required now. The durable control is a rule rather than a change: **anything rendered by a page is public, in the HTML and in the RSC payload alike**. Gated content needs real access control at a server, not merely the absence of a link. Worth stating explicitly if the site ever grows a members', dealer, or partner area.

API-SEC-03 Disclosure channel published without a confirmed owner

INFO

STANDARD	OWASP API	FORMAT	ACTION
RFC 9116	API9:2023 — Inventory Management	Valid — verified	Confirm mailbox

DESCRIPTION

Remediation published a `security.txt` nominating `security@renew.com` . The file itself is correct — it was validated field-by-field against RFC 9116:

```
True
True
True
True
True
True
```

IMPACT

The address was selected during remediation on the client's instruction, but **whether that mailbox exists and is monitored has not been confirmed by this assessment**. A published disclosure channel that routes to an unmonitored or non-existent address is arguably worse than none at all: a researcher who follows it in good faith and receives silence is markedly more likely to disclose publicly, and the organisation loses the private window it advertised.

RECOMMENDATION

- Confirm `security@renew.com` exists, is monitored, and reaches someone able to triage a security report.
- Set a calendar reminder before **17 August 2027**: an expired `Expires` field makes the file formally invalid under RFC 9116.
- Update the `Canonical` line when the site moves off the preview domain — alongside the CORS origin in `vercel.json` and `NEXT_PUBLIC_SITE_URL` .

5 OWASP API Security Top 10:2023 — Applicability

Each category assessed against the current application. Every entry marked not-applicable was **confirmed absent** per Section 2, not skipped. The final column is the one to read: it states what each category will require once the enquiry endpoint from Section 6 exists.

ID	CATEGORY	TODAY	WHY · AND WHAT IT BECOMES
API1	Broken Object Level Authorization	N/A	No objects, no identifiers, no per-user data. <i>Remains N/A</i> — an enquiry endpoint is write-only with no retrievable objects.
API2	Broken Authentication	N/A	No authentication mechanism exists. <i>Remains N/A</i> if the endpoint stays unauthenticated, which is appropriate for a public contact form.
API3	Broken Object Property Level Authorization	N/A	No object properties are exposed or accepted. <i>Becomes relevant</i> : reject unknown fields rather than binding them — see 6.2.
API4	Unrestricted Resource Consumption	N/A	Static files on a CDN; consumption is the platform's concern. <i>Becomes the single highest risk</i> — a public unauthenticated form is targeted by automation within days. See 6.3.
API5	Broken Function Level Authorization	N/A	No functions to invoke. <i>Remains N/A</i> for a single-purpose endpoint with no admin surface.
API6	Unrestricted Access to Sensitive Business Flows	N/A	No business flow is exposed. <i>Becomes relevant</i> : lead submission is a business flow worth abusing — spam, competitor noise, mailbox flooding. See 6.3.
API7	Server Side Request Forgery	N/A	No server, and no user-supplied value is ever fetched. <i>Stays N/A</i> provided the endpoint never fetches a user-supplied URL.
API8	Security Misconfiguration	FINDING	API-SEC-01 — build-time injection into generated XML, now fixed. <i>Expands</i> to headers, CORS and error handling on the new endpoint.
API9	Improper Inventory Management	INFO	API-SEC-02, API-SEC-03 — 46 undocumented data endpoints; a disclosure channel without a confirmed owner. <i>Expands</i> : the new endpoint must be documented and versioned.
API10	Unsafe Consumption of Third-Party APIs	N/A	The application consumes no external API — zero outbound calls. <i>Becomes relevant</i> the moment the endpoint forwards to a CRM or form backend. See 6.6.

READING THIS TABLE

Eight of ten categories are not applicable today, and that is a real and favourable result — it follows from an architecture with no server, which is the strongest security control this property has. It is not a clean bill of health for a future API. **Four categories change status the moment the enquiry endpoint ships**: API3, API4, API6 and API10. Those four are the substance of Section 6.

6 Control Specification — The Enquiry API

The enquiry-submission endpoint required to close VAPT-02 / SAST-03 / DAST-02 was deferred. When it is built it will be this property's first server-side attack surface and its first handler of personal data. This section specifies the controls to build in from the start.

6.1 Recommended shape

A **single-purpose Vercel Function** deployed alongside the static export — not a conversion of the site to server rendering. This keeps the server-side attack surface confined to one small, reviewable endpoint rather than extending it across every route, and it keeps the Next.js advisories tracked in VAPT-04 unreachable.

```
POST /api/enquiry
Content-Type: application/json

{ "name": string,           // required, 1-100
  "company": string,        // optional, 0-120
  "phone": string,          // required, E.164 or 10-digit Indian
  "email": string,          // required in modal flow, 0-254
  "state": string,          // required, must match the known state list
  "requirement": string,    // required, must match REQUIREMENT_TYPES
  "consent": true,          // required — see 6.5
  "website": ""             // honeypot — must be empty
}

201 { "ok": true }           // accepted
400 { "ok": false, "error": "invalid" } // validation failure, no detail
429 { "ok": false, "error": "rate" }   // rate limited
```

6.2 Input validation — API3

CONTROL	REQUIREMENT
Schema validation	Validate server-side against an explicit schema. Reject unknown fields rather than ignoring them — never spread the parsed body into a downstream object.
Length caps	Enforce on every field. Unbounded strings are the cheapest denial-of-service available.
Enum fields	<code>state</code> and <code>requirement</code> must be validated against the same closed lists the UI uses — the allow-list pattern already used well for the URL fragment.
Never trust the client	The dropdown fix delivered for SAST-02 is a usability control, not a security one. The server must independently reject a missing <code>requirement</code> .
Content-Type	Accept only <code>application/json</code> ; reject form-encoded bodies, which are what cross-origin form-based abuse uses.

6.3 Abuse resistance — API4 and API6

THE HIGHEST-RISK AREA OF THE NEW ENDPOINT

A public, unauthenticated form endpoint attached to a recognisable brand attracts automated submission quickly. The failure mode is not a breach — it is the sales team's inbox becoming unusable, which defeats the purpose of building the endpoint at all.

CONTROL	REQUIREMENT
Rate limiting	Per-IP limit (a handful per hour is generous for genuine enquiries) plus a global ceiling. Return <code>429</code> , never a stack trace.
Honeypot field	A hidden input real users never fill. Silently discard non-empty submissions — cheap, and it stops most unsophisticated bots.
Timing check	Reject submissions completed implausibly fast after page load.
CAPTCHA	Add when the above proves insufficient. Consider a privacy-respecting option given the DPDP context.
Payload size	Cap the request body — reject oversized bodies before parsing, not after.
Duplicate suppression	Debounce identical submissions to avoid a retry loop flooding the CRM.

6.4 Output handling — stored injection

Field values will travel onward into an email body, a CRM ticket, or an internal dashboard. Each of those is a different injection context, and the endpoint is the right place to think about all of them:

- **Email:** strip CR/LF from any value placed in a header — the classic header injection that turns a contact form into an open relay.
- **CRM / HTML dashboard:** encode on output. A name field containing markup becomes stored XSS against the staff member who opens the ticket.
- **CSV export:** a value beginning `=`, `+`, `-` or `@` is a formula injection vector when the export is opened in a spreadsheet. Prefix with an apostrophe.
- **Logs:** log for abuse detection, but exclude or mask the personal data fields.

6.5 Personal data — the coupled obligation**DO NOT SHIP THE ENDPOINT WITHOUT THIS**

VAPT-03 (privacy notice and consent) is currently accepted as an open risk purely because no personal data is transmitted or stored. The endpoint removes that condition. On the day it goes live, ReNew begins processing the personal data of Indian data principals, and the **Digital Personal Data Protection Act, 2023** obligations become active — notice at collection, stated purpose, consent, retention, withdrawal, and a grievance contact.

The privacy notice and consent control must ship **with or before** the endpoint, not after it. Treat them as one deliverable.

CONTROL	REQUIREMENT
Consent	Unticked checkbox, required, naming the purpose. Record the consent state with the submission.
Data residency	If a third-party form backend is chosen, confirm and document where the data is processed.
Retention	Define a period and an actual deletion process — not merely a policy sentence.
Transport & storage	TLS in transit; encryption at rest; access limited to the sales function.
Minimisation	Collect only what the sales conversation needs. Every extra field is extra liability.

6.6 Configuration and onward calls — API8 and API10

CONTROL	REQUIREMENT
CORS	Explicit origin allow-list on the endpoint. It must not inherit the site-wide value — and certainly not the wildcard replaced under VAPT-05.
Methods	Accept <code>POST</code> only; return <code>405</code> for everything else, matching the static site's current behaviour.
Secrets	CRM tokens in Vercel environment variables. Never committed — and never in a <code>NEXT_PUBLIC_</code> variable, which is inlined into client bundles and therefore public.
Env validation	Validate configuration at startup rather than interpolating it. This is the generalised lesson of API-SEC-01.
Error handling	Generic client responses; detail server-side only. No stack traces, no upstream error text echoed back.
Outbound calls (API10)	Validate the CRM response rather than trusting it. Set timeouts. Fail closed and surface a real error to the user — never a false success.
Fail-visible UI	Per DAST-02: the success state must derive from the server response. The current unconditional success flag is the defect that made the original finding High.

6.7 Pre-launch checklist

ITEM	VERIFIED BY
☐ Schema validation rejects unknown and oversized fields	Unit tests with hostile payloads
☐ Rate limit returns 429 under burst	Load a short burst; confirm throttling
☐ Honeypot submission silently discarded	Submit with the field populated
☐ CR/LF stripped from email-header-bound values	Submit a name containing <code>\r\n</code>
☐ Stored values encoded in the CRM view	Submit <code></code> ; open the ticket
☐ CSV export not formula-injectable	Submit a value starting <code>=</code> ; export and inspect
☐ CORS rejects a foreign origin	Cross-origin <code>fetch</code> from an unrelated page
☐ Non-POST methods return 405	Method enumeration, as in the DAST engagement
☐ UI shows a real error when the endpoint is down	Stop the endpoint; submit; confirm no false success
☐ Consent required; privacy notice live	Submit without consent — must be rejected
☐ Personal data absent from logs	Inspect logs after a test submission
☐ Secrets absent from the client bundle	<code>grep</code> the built output for token values

7 Appendix

7.1 Findings summary

ID	SEVERITY	FINDING	STATUS	OWASP API
API-SEC-01	LOW	Unvalidated env var injected into sitemap XML	FIXED	API8:2023
API-SEC-02	INFO	RSC payloads publicly enumerable	NO ACTION	API9:2023
API-SEC-03	INFO	Disclosure channel without a confirmed owner	CLIENT ACTION	API9:2023

7.2 Relationship to the other reports

REPORT	METHOD	CONTRIBUTION TO THE API QUESTION
VAPT v1.1	Black-box + source	First established API absence; §3.2 is the original determination
SAST v1.1	Static analysis	Confirmed zero network calls and zero server-side entry points in 6,689 lines
DAST v1.1	Runtime	Confirmed zero egress at runtime with instrumented <code>fetch /XHR/beacon</code> ; found the RSC endpoints
API v1.0	Endpoint-focused	Re-verified absence post-remediation; assessed the 49 data endpoints; specified controls for the deferred endpoint

All four reports describe the same target on the same date. Severities are consistent across the set; each underlying defect should be remediated once.

7.3 Method

- **Static:** exhaustive search for route handlers, middleware, server actions, API directories and outbound-call constructs across the full source tree.
- **Build:** route table inspection; enumeration of every non-HTML artifact in the production output.
- **Dynamic:** per-endpoint content-type and size inspection; RSC-vs-HTML disclosure comparison; RFC 9116 field validation; sitemap URL resolution against build artifacts; adversarial build with a hostile environment variable.
- **Standards:** OWASP API Security Top 10:2023, RFC 9116, CWE.

7.4 Limitations

- **Point-in-time**, reflecting commit `a4f4020` plus the remediation applied 17 August 2026.
- **No API existed to test.** Section 6 is a specification, not an assessment — it cannot substitute for testing the endpoint once built. **A focused API penetration test is recommended after it ships.**

- **Deploy-dependent controls unverified.** The headers in `vercel.json` — including `nosniff` on these data endpoints — take effect only on deployment, which was not performed.
- **Mailbox ownership unconfirmed** for the published security contact (API-SEC-03).

END OF REPORT

ReNew Solar Marketing Website — API Security Assessment v1.0 — 17 August 2026. Application APIs: 0 (verified). Data endpoints assessed: 49. 3 findings: 0 Critical, 0 High, 0 Medium, 1 Low, 2 Informational. 1 fixed during assessment. 8 of 10 OWASP API categories not applicable; 4 change status when the enquiry endpoint ships.